

Raft under Physical Communication Models: What Relativity Changes, and What It Does Not

Tomasz Kazulak
Independent study

Structured literature survey · 2026-09-26

Abstract. Consensus protocols such as Raft are specified against abstract communication models, while physical systems add constraints those models do not name: a finite signal speed, clocks that measure proper time, and no observer-independent simultaneity. We survey 41 studies, identified by targeted searches and backward and forward citation chasing and checked against their full texts, to establish what is known about Raft under non-ideal and physically grounded communication models. Three findings emerge. First, Raft’s safety depends on no timing assumption and has recently been proved to survive relativistic causality. Second, the large literature on Raft under loss, partitions and variable delay concerns liveness and performance at the protocol level, and relativity adds nothing to it beyond a lower bound and a time dependence on delay. Third, the one Raft mechanism whose safety depends on clocks, the leader lease, has been analysed only against a common reference time: a bound on relative clock drift, a bounded uncertainty around true time, or timestamps whose relation to real time is delegated to another layer. None of these is stated frame-invariantly, and no study examines leases, or clock-based coordination generally, in relativistic spacetime. We identify this as the open problem and state candidate questions for an analytical follow-up.

1 Introduction

Distributed-systems courses present consensus against an idealised network in which messages are delayed arbitrarily but eventually delivered, and processes share no clock [1], [2]. A physicist reading these models naturally asks whether they survive contact with the physical world. Signals propagate no faster than light. Clocks on moving or gravitationally separated nodes accumulate different proper times [3], [4]. And relativity denies a global notion of “now”, on which specifications such as linearizability [5] depend: their definition assumes a total temporal order, “tantamount to assuming the existence of a global clock” [6]. Lamport’s logical clocks were themselves motivated by special relativity [7], yet whether a concrete, widely deployed protocol stays correct under physical constraints has only recently been proved, for Raft [8], building on relativistic correctness conditions of Gilbert and Golab [6] and Jayanti [9].

We take Raft [10] as the object of study. It is widely deployed and precisely specified, and it separates cleanly into a timing-independent safety core and timing-dependent mechanisms. The survey’s aim is to fix the boundaries of current knowledge before any new analysis is attempted: what has been established, what follows directly from established models, and what is open.

The contributions are (i) a map of the communication models under which Raft and equivalent protocols have been analysed; (ii) a classification of Raft’s timing-dependent mechanisms by whether they affect safety or only liveness; (iii) a synthesis of the small literature on distributed computing in relativistic spacetime; and (iv) the identification of clock-based coordination under relativistic time as the open problem.

2 Scope and research questions

The object is Raft and leader-based crash-fault-tolerant state-machine replication whose results transfer to Raft. We consider communication models ordered by the constraints they impose:

- synchronous;
- partially synchronous [2];
- asynchronous [1];

- asynchronous with loss, omission and partial partitions;
- delay- and disruption-tolerant networks [11];
- relativistic spacetime, in which delivery respects the light-cone structure and each process measures its own proper time.

Out of scope are time-transfer engineering without a computational model, quantum communication, and blockchain systems that use Raft as a black box.

RQ1 Under which communication models have Raft and equivalent protocols been analysed, for which properties, and with which methods?

RQ2 Which Raft mechanisms depend on timing or clock assumptions, for safety or only for liveness, and how are those assumptions stated?

RQ3 Which models of distributed computation in relativistic spacetime exist, and what correctness results do they establish?

RQ4 To what extent have clock-dependent mechanisms been analysed under relativistic or otherwise physically grounded models of time?

3 Method

This is a structured survey rather than a full systematic review. It follows the reporting logic of PRISMA 2020 [12] without claiming its complete procedure, and it uses snowballing as described by Wohlin [13].

Identification. Sixteen targeted web-search queries produced 149 records. A first pass removed non-scholarly pages and duplicates, screened 73 records by title and abstract, and retained 34. Backward citation chasing from these and from the Raft literature added 19 records. These two counts come from the search log of that first pass; the records it excluded were not re-screened.

Forward snowballing. Eight seeds were chosen as the studies most central to the question that also have few enough citations to screen completely. Five come from the relativistic and timekeeping strand [4], [6], [8], [9], [14], and three concern Raft under network faults and leases [15], [16], [17]. Litmaps returned 56 citing records. Merging preprint and published versions and repeated entries leaves 39 distinct works, of which 5 had already been identified. A ninth intended seed, Mattern [18], is not indexed by Litmaps.

Additional citation chasing. Because the relativistic strand is small and central, two further checks were run. Backward: the reference lists of Amini and Golab [8], Gilbert and Golab [6] and Jayanti [9] contributed 13 records not already identified. Forward: every work citing Gilbert and Golab or Mattern was retrieved from OpenAlex and Semantic Scholar, since Google Scholar cannot be queried by script, contributing 20 records not already identified. Together these added two studies [19], [20].

Screening and eligibility. Each record was first tagged as background (foundational or method references cited for context, such as FLP, DLS, linearizability, PRISMA and the lease and failure-detector classics) or as a study; only studies were screened. A study was included if its subject is Raft or leader-based crash-fault replication whose results transfer to Raft, a model of distributed computation in relativistic spacetime, or quantified relativistic or physical timekeeping in distributed systems; if an explicit communication, timing or clock model is central to it; if it makes a technical contribution; and if it is in English. Exclusion reasons were Raft as a black-box orderer, Byzantine-only protocols, physics without a distributed-system model, non-scholarly or insufficient detail, duplicates, and unavailable full text. Title and abstract screening covered 103 distinct studies. Full texts were obtained for 42 of the 47 retained, from open-access sources, the authors' sites, or the author's institutional access; the other 5 were excluded as unavailable. Full-text eligibility removed one more, leaving 41 studies, each summarised on an extraction card (`review/cards.md`) and listed with every decision in `review/studies.csv`. Table 1 summarises the flow.

Table 1: Study selection. Counts for the first search pass are taken from its log; all later counts are from `review/studies.csv`.

Stage	Records
Targeted searches (first pass): retained	34
Backward citation chasing (first pass)	19
tagged background, not screened	12
Forward snowballing via Litmaps: citing records (8 seeds)	56
duplicates and versions merged	17
already identified	5
Additional backward chasing (3 reference lists)	13
tagged background, not screened	2
Additional forward chasing (2 seeds, OpenAlex and Semantic Scholar)	20
duplicates and versions merged	3
Studies screened on title and abstract	103
excluded	56
Full texts sought	47
not retrievable	5
Full texts assessed	42
excluded	1
Studies included	41

4 Use of AI assistance

This survey was conducted with an AI assistant, Claude (Anthropic), in two modes: through a chat interface connected to web search, the author’s Zotero library and arXiv, and as Claude Code working directly in the project repository. We report the division of work because it bears directly on how far the findings can be trusted.

Performed by the assistant in chat:

- proposing the research questions, search strings and eligibility criteria;
- running the targeted searches and the first screening pass;
- resolving bibliographic metadata and adding records to Zotero;
- drafting the text of this article;
- the preliminary lease argument in Section 6 and a numerical check of it.

Performed by Claude Code:

- tagging records as background or study, merging preprint and published versions, and attributing forward-snowballing records to their seeds using citation data from OpenAlex and Semantic Scholar;
- title and abstract screening of all studies against the stated criteria, with abstracts fetched by DOI or arXiv identifier where Zotero lacked them;
- retrieving full texts, from open-access sources and, with the author’s permission, through the author’s browser session, and assessing full-text eligibility;
- the additional backward and forward citation chasing described in the Method;
- checking the authors, title and year of every bibliography entry that has a DOI or arXiv identifier against Crossref or arXiv;
- extracting data from the full texts into one card per included study, with page references;
- checking every sentence of this article that attributes a result to a cited work against the cards, and correcting or removing the sentences that the full texts did not support.

Performed by the author:

- posing the motivating question and deciding the scope;
- choosing to reduce a full systematic review to a structured survey;

- confirming the snowballing seeds and running the forward chase in Litmaps, a citation index rather than a generative tool;
- supplying full texts that could not be retrieved automatically;
- approving the screening and the full-text eligibility decisions at two checkpoints, and taking responsibility for the result.

Verification status. Claims about included studies were checked against their full texts; where only a preprint was openly available, the preprint was used. Background references were not checked against full texts. Every bibliography entry with a DOI or arXiv identifier matches the registry’s authors; entries without one were checked against the title pages of the retrieved texts. Metadata that the chat assistant took from search snippets and that no identifier confirms should be treated as unverified. An earlier reference file listed four authors for Aeini and Golab [8], which has two.

The principal risks are misattributed or overstated claims, screening errors that a second human reviewer would catch, and a uniform framing inherited from a single drafting voice. Claude Code applied the eligibility criteria, and the author reviewed its decisions rather than screening independently; approving decisions made by someone else invites automation bias. An independent blind check by the author, re-screening 20 randomly drawn records (`review/blind-check.csv`, seed 20260926) without seeing the recorded decisions, is prepared and its agreement rate is to be reported here.

5 Results

5.1 RQ1: Communication models and methods

Table 2 maps the corpus. The division of labour is sharp. Protocol safety is analysed by proof in the asynchronous model: Raft’s TLA+ specification lets the network reorder, drop and duplicate messages, and State Machine Safety is proved against it by hand [21]. A machine-checked proof in Coq establishes the same property, and end-to-end linearizable replication, for an implementation under a network semantics with drops, duplication and reordering; it covers safety only [22]. A Raft-derived protocol has been model checked, with implementation traces validated against the specification, under the same kind of network abstraction [23]. Almost everything else, from lossy links to wireless and space deployments, is analysed for liveness, latency or availability, by simulation, analytical modelling or testbeds. The exceptions concern implementations rather than the protocol: real systems violate safety under network partitions, for instance by letting an isolated leader serve stale reads [24], and model checking has found safety bugs where an implementation diverged from Raft [23].

Table 2: Communication models under which Raft and equivalent protocols have been analysed.

Model	What is analysed	Studies
Asynchronous; loss, duplication, reordering	Safety of the time-free core: TLA+ specification with hand proof; machine-checked proof in Coq; model checking and trace validation of a Raft variant; safety checks on simulation traces	[10], [21], [22], [23], [25]
Partially synchronous	Liveness of leader election under the timing requirement that broadcast time be much less than the election timeout; election latency on LAN and simulated WAN	[10], [21], [25], [26]
Partial and asymmetric connectivity	Repeated elections and livelock under partial partitions; effect of PreVote, CheckQuorum and adaptive timeouts; real-world partition failures	[15], [16], [23], [24], [26], [27]
Unreliable channels	Which channel failures consensus can tolerate; tight connectivity bounds	[28], [29]
Probabilistic loss and delay	Split probability, response time, availability and reliability as functions of loss rate, failure rates and timeouts	[30], [31], [32], [33]
Variable wide-area delay	Adaptive or learned election timeouts; leader placement; randomised protocols whose liveness does not rest on timeouts	[34], [35], [36], [37], [38]
Wireless, edge, space	Raft variants and evaluations over vehicular, millimetre-wave, edge and satellite links; quorum design over light-time Earth–Moon–Mars delays	[39], [40], [41], [42], [43]
Relativistic spacetime	Relativistic linearizability of Raft under physical causality; transfer of asynchronous correctness to relativistic executions	[8], [9]

5.2 RQ2: Timing-dependent mechanisms

Raft uses time in four places (Table 3). Election timeouts, heartbeats, and the PreVote and CheckQuorum extensions affect only availability. Raft requires that “safety must not depend on timing” [21] and ensures at most one leader per term [10]; a badly tuned timeout causes needless or failed elections [21]. How PreVote and CheckQuorum behave under partial connectivity depends on the shape of the partition. PreVote stopped the repeated elections in one emulated partial partition [15] but does not prevent every disruptive-server case [21]. With PreVote and CheckQuorum enabled, Raft recovers from the quorum-loss and chained scenarios but stays unavailable for the whole partition when the only quorum-connected server has an outdated log, because a Raft leader must also hold the most up-to-date log [16]. CheckQuorum can leave a well-connected node’s term lagging, so that recovery takes many election rounds [27]. A leader that cannot receive messages can keep followers from timing out [23].

The exception is the leader lease used to serve linearizable reads without a quorum round-trip. The Raft thesis extends the lease to the start of a heartbeat round plus the election timeout divided by a clock drift bound, and states the assumption as a bound on relative drift: over a given time period, no server’s clock advances more than this bound times any other. If it is violated the system “could return arbitrarily stale information” [21].

Recent lease designs state the clock assumption differently. LeaseGuard assumes clocks with bounded uncertainty, each returning an interval guaranteed to contain the true time, and loses linearizability if those bounds are wrong [17]. T-Lease models synchronised clocks with a maximum drift rate against a global time and treats an attacker who changes clock value or frequency as the threat [44]. CockroachDB’s Leader Leases rely only on monotonic local clocks, carry hybrid-logical-clock timestamps on every message, and prove that leases are disjoint over timestamps “on which both [nodes] agree, although in real time the two nodes reach that timestamp at different times”. The link from those timestamps to real-time consistency is delegated to a transaction layer that assumes loosely synchronised clocks [45]. ReadIndex, the quorum-based alternative, confirms leadership with one round of heartbeats to a majority and uses no clock [21].

Table 3: Where Raft depends on time.

Mechanism	Affects	Timing assumption as stated
Election timeout, heartbeat	Liveness	$\text{broadcastTime} \ll \text{electionTimeout} \ll \text{MTBF}$ [21]
PreVote, CheckQuorum	Liveness	As above; outcome under partial connectivity depends on partition shape
Leader lease (lease reads)	Safety	Bounded relative clock drift [21]; bounded uncertainty around true time [17]; maximum drift rate [44]; disjointness over hybrid-logical-clock timestamps, real time delegated [45]
ReadIndex reads	Neither	None (one round of heartbeats to a majority)

5.3 RQ3: Distributed computing in relativistic spacetime

The strand is small, continuous and recent. Lamport’s formalism for interprocess communication already admitted executions whose events are points of four-dimensional spacetime ordered by the happens-before relation of special relativity, next to the more familiar global-time models [19]. Mattern observed that vector time in distributed systems has the structure of Minkowski causality: for two processes, vector-time order and light-cone order are basically identical, and causality-preserving deformations of time diagrams correspond to Lorentz transformations [18]. Matherat and Jaekel argued that synchronous circuits rely on the simultaneity classes of Newtonian space-time and asynchronous circuits on the causal structure of relativistic space-time [14]. They later generalised the trace formalism for delay-insensitive circuits to relativistic causality [46].

Gilbert and Golab defined four relativistic variants of linearizability that need no global clock and form a hierarchy: R0, the direct generalisation to partially ordered events; R1, linearizable in some frame of reference; R2, in every frame; and R3, in every frame with a single common linearization. Only R2 is local, which is why they consider it the most natural definition. R3 is the strongest condition, is easier to reason about, and is not local. They also give sufficient conditions for deducing R2 and R3 from R1, and remark, without a full proof, that Paxos state machine replication satisfies them and so yields R3 [6]. Jayanti’s central theorem relates classical, relativistic and purely computational executions of asynchronous algorithms: an algorithm satisfies a property classically if and only if it does in every observer’s frame, so every linearizable algorithm is relativistically linearizable. His definition differs slightly from R2 by admitting only orderings that correspond to actual reference frames, and his model has no clocks [9]. Amini and Golab proved R3 for any execution of Raft, taking the final leader’s log order as the single linearization [8]. Their model treats every operation as a log command, so read optimisations are outside it.

Forward chasing from all seeds and the additional chasing found one further work. It frames consistency models as observers of a causal graph and argues that linearizability needs a global serializer because a single order over spacelike-separated operations is “a preferred frame” no observer’s light cone supplies. Its relativistic reading is explicitly interpretive; the model itself uses Newtonian real time [20].

A separate engineering literature quantifies relativistic timekeeping. It gives closed-form models of relativistic clock rate and synchronisation for system designers, and notes that some distributed computing needs time intervals rather than ordering alone [4]. In GPS, the net rate offset of a satellite

clock is about 4.46×10^{-10} , and clocks are synchronised by convention in an Earth-centred inertial frame [3]. For relativistic spacecraft, the query–response latency seen on one node’s clock follows from the trajectories [47]. Clock drift between two nodes raises the average age of information, in closed form [48]. None of it contains a model of distributed coordination.

5.4 RQ4: Clock-based coordination under physical time

No study in the corpus analyses a clock-dependent consensus mechanism under a relativistic model of time. The relativistic results cover asynchronous algorithms, whose correctness makes no reference to clocks; Jayanti’s model has no clocks at all [9]. Aeni and Golab explicitly name algorithms that use physical clocks for coordination as future work [8]. A general condition for when ordering by physical clocks respects causality, namely that timestamp order refines causality if and only if twice the clock-error bound does not exceed the minimum cross-node latency, is stated against Newtonian real time [20]. The lease studies, conversely, reason against a common reference time: relative drift [21], uncertainty around true time [17], or a drift rate against a global clock [44]. The closest contact is CockroachDB’s design, which proves lease disjointness over timestamps that nodes agree on while reaching them at different real times, but leaves the step from timestamps to real-time consistency to a layer that assumes loosely synchronised clocks [45]. The timekeeping studies have no protocol model. The two literatures do not meet.

6 Discussion

We separate what the surveyed studies establish from our own reasoning. Only the first list below reports findings of the literature; everything after it is the authors’ reasoning or conjecture, is not a result of this survey, and is marked as such.

Established by the surveyed studies.

- Raft’s safety is independent of timing [21], is machine-checked for an implementation [22], and holds under relativistic causality [8], [9].
- Its liveness requires the timing requirement $\text{broadcastTime} \ll \text{electionTimeout} \ll \text{MTBF}$ [21], and fails in specific ways under partial and asymmetric connectivity [15], [16], [23], [27].
- Its only timing-dependent safety mechanism is the leader lease [21].

Our reasoning, not a finding of the survey. Three consequences seem to follow from the models above without new results.

- A network that delivers messages only into the sender’s future light cone produces a subset of asynchronous executions, so every asynchronous safety argument transfers. Jayanti’s equivalence theorem proves a general form of this for asynchronous algorithms [9].
- Physics bounds message delay from below, by distance over c , and makes it time-varying. This instantiates partial synchrony rather than replacing it, so for liveness relativity is “just delay”.
- Table 4 compares the sizes involved. The largest motion-induced effect is first order in $\beta = v/c$ and is kinematic: a changing propagation delay. Genuinely relativistic rate effects are second order, or of order $\Delta\Phi/c^2$ for gravity, and lie orders of magnitude below the tolerance of commodity oscillators.

Table 4: Orders of magnitude of rate effects relevant to timeouts and leases.

Effect	Order	Typical size
Changing propagation delay (first-order Doppler)	β	5×10^{-5} between LEO satellites; 10^{-4} at Earth’s orbital speed
Time dilation	$\beta^2/2$	$\approx 3 \times 10^{-10}$ in low Earth orbit
Net rate offset, GPS orbit vs. geoid (gravitational blueshift less time dilation)	$\Delta\Phi/c^2, \beta^2/2$	$\approx 4.46 \times 10^{-10}$ [3]
Commodity quartz oscillator tolerance	—	10^{-5} to 10^{-4}

Open (our interpretation of the gap). What does not reduce to delay is the relativity of simultaneity, and it bites exactly where a protocol converts elapsed time on one clock into a claim about another node. “Bounded drift between two clocks” presupposes a simultaneity convention for

comparing distant clocks, as do “uncertainty around true time” and “drift against a global clock”. None of these is a frame-invariant assumption. CockroachDB’s timestamp-domain disjointness comes closest to avoiding the convention, but it still needs synchronised clocks to turn disjoint timestamps into fresh reads [45]. The lease safety argument therefore has no frame-invariant statement in the literature.

Conjecture (ours, unverified; not a finding of this survey). A preliminary observation, to be verified rather than relied upon, suggests what such a statement might look like. The causal chain that could make a lease read stale passes through the follower’s timer. By the reverse triangle inequality of Minkowski spacetime, an inertial leader’s proper time along that chain is at least the follower’s timeout. If so, first-order Doppler effects cancel, and only a leader whose worldline is not a geodesic, whether accelerating or held static in a gravitational field, can lose lease time: a twin-paradox effect.

7 Threats to validity

The gap was a hypothesis before the search. The lease question was proposed by the chat assistant in its first answer, before any screening, and it shaped the search terms, the eligibility criteria and the choice of snowballing seeds. The survey was therefore aimed at the gap it reports, and a reader should weigh the RQ4 conclusion accordingly. Independent support comes from Aeini and Golab, who name clock-based algorithms as future work in their own words [8].

Screening. The survey had a single human reviewer, and screening, extraction and drafting were AI-assisted (Section 4). The author approved decisions made by the AI rather than making them, which invites automation bias; the blind re-screening described in Section 4 is the check on this.

Identification. Identification relied on targeted web search rather than on Scopus or Web of Science, and the queries were partly seeded with known items, which biases toward known work. Only counts survive from the 149 records of the first search pass, not the records themselves, so that pass cannot be reproduced and its exclusions were not re-screened. Forward snowballing depends on Litmaps’ index, which does not cover one intended seed; for that seed and for Gilbert and Golab, citing works were taken from OpenAlex and Semantic Scholar instead of Google Scholar, and those indexes may miss citations from venues they cover poorly. The relativistic strand is small enough that we believe the combined chase covers it. The Raft-under-faults strand is large, and it is represented rather than exhausted.

Retrieval. Five studies were excluded only because their full texts could not be obtained; none is central to the research questions. Where only a preprint was available, it was used in place of the published version.

8 Conclusion and next steps

Physics changes little about Raft. Its safety core survives relativistic causality, and its liveness sees physics only as delay. The single point of contact is the leader lease, the one mechanism that turns local elapsed time into a claim about other nodes. Its safety assumption has never been stated in frame-invariant form; the nearest approach, disjointness over agreed timestamps, still relies on synchronised clocks to make reads fresh. We take the following candidate questions to the analytical phase:

1. What is the frame-invariant condition under which a Raft leader lease is safe?
2. In which physically realistic configurations, such as satellites, ground-orbit links and interplanetary distances, does that condition differ measurably from the classical bounded-drift rule?
3. Does a lease-based read satisfy the strongest relativistic linearizability condition that Raft satisfies without leases [8]?

References

- [1] M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of Distributed Consensus with One Faulty Process,” *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, Apr. 1985, doi: 10.1145/3149.214121.

- [2] C. Dwork, N. Lynch, and L. Stockmeyer, “Consensus in the Presence of Partial Synchrony,” *Journal of the ACM*, vol. 35, no. 2, pp. 288–323, Apr. 1988, doi: 10.1145/42282.42283.
- [3] N. Ashby, “Relativity in the Global Positioning System,” *Living Reviews in Relativity*, vol. 6, no. 1, p. 1, Jan. 2003, doi: 10.12942/lrr-2003-1.
- [4] D. G. Messerschmitt, “Relativistic Timekeeping, Motion, and Gravity in Distributed Systems,” *Proceedings of the IEEE*, vol. 105, no. 8, pp. 1511–1573, 2017, doi: 10.1109/JPROC.2017.2717980.
- [5] M. P. Herlihy and J. M. Wing, “Linearizability: A Correctness Condition for Concurrent Objects,” *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 3, pp. 463–492, Jul. 1990, doi: 10.1145/78969.78972.
- [6] S. Gilbert and W. Golab, “Making Sense of Relativistic Distributed Systems,” Springer Berlin Heidelberg, 2014, pp. 361–375. doi: 10.1007/978-3-662-45174-8_25.
- [7] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978, doi: 10.1145/359545.359563.
- [8] K. Aeni and W. Golab, “Analyzing Linearizability in Relativistic Distributed Systems,” *ACM*, Jul. 2026, pp. 1–9. doi: 10.1145/3820355.3820388.
- [9] S. V. Jayanti, “On Interplanetary and Relativistic Distributed Computing,” *ACM*, Jun. 2025, pp. 192–202. doi: 10.1145/3732772.3733563.
- [10] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” in *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*, 2014. [Online]. Available: <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [11] S. Burleigh *et al.*, “Delay-Tolerant Networking: An Approach to Interplanetary Internet,” *IEEE Communications Magazine*, vol. 41, no. 6, pp. 128–136, Jun. 2003, doi: 10.1109/MCOM.2003.1204759.
- [12] M. J. Page *et al.*, “The PRISMA 2020 Statement: An Updated Guideline for Reporting Systematic Reviews,” *BMJ*, p. n71, Mar. 2021, doi: 10.1136/bmj.n71.
- [13] C. Wohlin, “Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering,” *ACM*, May 2014, pp. 1–10. doi: 10.1145/2601248.2601268.
- [14] P. Matherat and M.-T. Jaekel, “Concurrent Computing Machines and Physical Space-Time.” [Online]. Available: <https://arxiv.org/abs/cs/0112020>
- [15] C. Jensen, H. Howard, and R. Mortier, “Examining Raft’s Behaviour during Partial Network Failures,” *ACM*, Apr. 2021, pp. 11–17. doi: 10.1145/3447851.3458739.
- [16] H. Ng, S. Haridi, and P. Carbone, “Omni-Paxos: Breaking the Barriers of Partial Connectivity,” *ACM*, May 2023, pp. 314–330. doi: 10.1145/3552326.3587441.
- [17] A. J. J. Davis, M. Demirbas, and L. Deng, “LeaseGuard: Raft Leases Done Right.” [Online]. Available: <https://arxiv.org/abs/2512.15659>
- [18] F. Mattern, “On the Relativistic Structure of Logical Time in Distributed Systems,” in *Parallel and Distributed Algorithms*, Elsevier Science Publishers, 1992, pp. 215–226.
- [19] L. Lamport, “On Interprocess Communication. Part I: Basic Formalism,” *Distributed Computing*, vol. 1, no. 2, pp. 77–85, 1986, doi: 10.1007/BF01786227.
- [20] R. Landers and K. Kramer, “Light Cone Consistency: Closure, Ordering, and the Single-Observer Boundary.” [Online]. Available: <https://arxiv.org/abs/2605.09114>
- [21] D. Ongaro, “Consensus: Bridging Theory and Practice,” phdthesis, 2014. [Online]. Available: <https://web.stanford.edu/~ouster/cgi-bin/papers/OngaroPhD.pdf>
- [22] D. Woos, J. R. Wilcox, S. Anton, Z. Tatlock, M. D. Ernst, and T. Anderson, “Planning for Change in a Formal Verification of the Raft Consensus Protocol,” *ACM*, Jan. 2016, pp. 154–165. doi: 10.1145/2854065.2854081.
- [23] H. Howard, M. A. Kuppe, E. Ashton, A. Chamayou, and N. Crooks, “Smart Casual Verification of the Confidential Consortium Framework,” in *22nd USENIX Symposium on Networked Systems*

- Design and Implementation (NSDI 25)*, Philadelphia, PA: USENIX Association, Apr. 2025. [Online]. Available: <https://www.usenix.org/conference/nsdi25/presentation/howard>
- [24] A. Alquraan, H. Takruri, M. Alfatafta, and S. Al-Kiswany, “An Analysis of Network-Partitioning Failures in Cloud Systems,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018.
 - [25] H. Howard, M. Schwarzkopf, A. Madhavapeddy, and J. Crowcroft, “Raft Refloated: Do We Have Consensus?,” *ACM SIGOPS Operating Systems Review*, vol. 49, no. 1, pp. 12–21, Jan. 2015, doi: 10.1145/2723872.2723876.
 - [26] H. Howard, “ARC: Analysis of Raft Consensus,” 2014. [Online]. Available: <https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-857.pdf>
 - [27] Z. Liang, V. Jabrayilov, A. Charapko, and A. Aghayev, “MultiPaxos Made Complete.” [Online]. Available: <https://arxiv.org/abs/2405.11183>
 - [28] A. Naser-Pastoriza, G. Chockler, and A. Gotsman, “Fault-Tolerant Computing with Unreliable Channels (Extended Version).” [Online]. Available: <https://arxiv.org/abs/2305.15150>
 - [29] A. Naser-Pastoriza, G. Chockler, A. Gotsman, and F. Ryabinin, “Tight Bounds on Channel Reliability via Generalized Quorum Systems,” *ACM*, Jun. 2025, pp. 444–454. doi: 10.1145/3732772.3733529.
 - [30] D. Huang, X. Ma, and S. Zhang, “Performance Analysis of the Raft Consensus Algorithm for Private Blockchains.” [Online]. Available: <https://arxiv.org/abs/1808.01081>
 - [31] Y. Li, Y. Fan, L. Zhang, and J. Crowcroft, “RAFT Consensus Reliability in Wireless Networks: Probabilistic Analysis,” *IEEE Internet of Things Journal*, vol. 10, no. 14, pp. 12839–12853, Jul. 2023, doi: 10.1109/JIOT.2023.3257402.
 - [32] Y. Li, Z. Xu, Z. Zhou, Y. Wang, L. Zhang, and J. Crowcroft, “Distributed Consensus Network: A Modularized Communication Framework and Reliability Probabilistic Analysis,” *IEEE Transactions on Networking*, vol. 34, pp. 4084–4095, 2026, doi: 10.1109/TON.2026.3667834.
 - [33] E. Sakic and W. Kellerer, “Response Time and Availability Study of RAFT Consensus in Distributed SDN Control Plane.” [Online]. Available: <https://arxiv.org/abs/1902.02537>
 - [34] K. Shiozaki and J. Nakamura, “Dynamic Tuning of Election Parameters for Timely Leader Failover in State Machine Replication.” [Online]. Available: <https://arxiv.org/abs/2507.15154>
 - [35] Q. Wang, “BALLAST: Bandit-Assisted Learning for Latency-Aware Stable Timeouts in Raft.” [Online]. Available: <https://arxiv.org/abs/2512.21165>
 - [36] Y. Wang, Z. Cheng, Y. Dong, and Z. Xu, “CD-Raft: Reducing the Latency of Distributed Consensus in Cross-Domain Sites.” [Online]. Available: <https://arxiv.org/abs/2603.10555>
 - [37] P. Tennage *et al.*, “QuePaxa: Escaping the Tyranny of Timeouts in Consensus,” *ACM*, Oct. 2023, pp. 281–297. doi: 10.1145/3600006.3613150.
 - [38] P. Tennage, A. Desjardins, and L. Kokoris-Kogias, “RACS-SADL: Robust and Understandable Randomized Consensus in the Cloud,” *IEEE*, Jul. 2025, pp. 362–373. doi: 10.1109/cloud67622.2025.00044.
 - [39] Z. Hu *et al.*, “RUBICONE: Wireless RAFT-Unified Behaviors for Intervehicular Cooperative Operations and Negotiations.” [Online]. Available: <https://arxiv.org/abs/2603.18595>
 - [40] H. Luo, X. Yang, H. Yu, G. Sun, B. Lei, and M. Guizani, “Performance Analysis and Comparison of Non-ideal Wireless PBFT and RAFT Consensus Networks in 6G Communications.” [Online]. Available: <https://arxiv.org/abs/2304.08697>
 - [41] A. Jeffery, H. Howard, and R. Mortier, “Mutating Etcd Towards Edge Suitability.” [Online]. Available: <https://arxiv.org/abs/2311.09929>
 - [42] J. Li *et al.*, “Satellite Network-Optimized Dynamic Scoped Hierarchical Raft for Blockchain Consensus,” *Autonomous Intelligent Systems*, vol. 6, no. 1, Mar. 2026, doi: 10.1007/s43684-026-00126-3.
 - [43] T. Mason, “Legible Consensus: Topology-Aware Quorum Geometry for Asymmetric Networks.” [Online]. Available: <https://arxiv.org/abs/2603.28788>

- [44] B. Trach, R. Faqeh, O. Oleksenko, W. Ozga, P. Bhatotia, and C. Fetzer, “T-Lease: A Trusted Lease Primitive for Distributed Systems.” [Online]. Available: <https://arxiv.org/abs/2101.06485>
- [45] I. Kettaneh *et al.*, “Scalable Leader Leases For Multi Consensus Groups in CockroachDB,” ACM, May 2026, pp. 373–385. doi: 10.1145/3788853.3803081.
- [46] P. Matherat and M.-T. Jaekel, “Relativistic Causality and Clockless Circuits.” [Online]. Available: <https://arxiv.org/abs/1112.5200>
- [47] D. Messerschmitt, I. Morrison, T. Mozdzen, and P. Lubin, “Timing Relationships and Resulting Communications Challenges in Relativistic Travel.” [Online]. Available: <https://arxiv.org/abs/2311.14039>
- [48] M. Salimnejad, N. Pappas, and M. Kountouris, “So Timely, Yet So Stale: The Impact of Clock Drift in Real-Time Systems.” [Online]. Available: <https://arxiv.org/abs/2501.00549>